

Exercícios — Módulo 9: Projeto CRUD

Completo + Tratamento de Erros

Aula 9 · 29/07 · Jornada DEV START — Programa START (TOTVS Paulista) **Prazo de entrega:** até **segunda, 03/08**. O curso encerra em 29/07 — você tem o fim de semana. Só os exercícios marcados com  são obrigatórios; os demais são bônus, sem nota. Suporte no Discord até 07/08. **Onde entregar:** na **atividade do Módulo 9 no Google Classroom**. Suba os fontes (`.PRW`) e os prints das telas no seu **GitHub** e cole o link do repositório na atividade. Depois, comemore postando um print do sistema rodando no Discord `#conquistas` . 

O que é obrigatório aqui

Estamos na **reta final** (última aula em 29/07). Por isso, o **obrigatório são 2 exercícios**: o **5** (tratamento de erros, roda em **Harbour puro** — não precisa de Protheus) e o **6** (gravação segura). Os exercícios 1, 2, 3, 4 e 7 continuam aqui completos e também estão no **pacote bônus** (`bonus-projeto-2-tabelas.md` , com as tabelas SZ2/SZ3): **sem nota, sem prazo**, com suporte no Discord até 07/08.

Antes de começar

- Estes exercícios constroem **um projeto único**: o sistema de Contatos (SZ1) e Interações (SZ2). Faça na ordem — cada exercício depende do anterior.
- Trabalhe no **Protheus** (DevStudio/TDS). Compile a cada etapa e teste antes de seguir.
- O objetivo não é só “funcionar”, é **funcionar como um sistema profissional**: com validações, integridade e **tratamento de erros**.
- Travou? O canal é o `#duvidas-parte2-protheus` . 

Exercício 1 — Dicionário de dados completo (*bônus — virou material de autoestudo*)

Configure no dicionário as **duas tabelas** do projeto:

- **SZ2** e **SZ1** no **SX2** (modo Compartilhado)
- Todos os **campos** no **SX3** (respeite tipo, tamanho e contexto Real/Virtual da apostila)
- Os **índices** no **SIX**:
 - SZ1 ordem 1: `Z1_FILIAL + Z1_CODIGO`
 - SZ2 ordem 1: `Z2_FILIAL + Z2_CONTAT + Z2_SEQUEN`
- O **domínio SX5** código `Z2` com os tipos de interação (E, L, R, V, W)

🔥 **Entrega:** prints do Configurador (SX2, SX3, SIX, SX5) no repositório.

Exercício 2 — Biblioteca e rotinas (*bônus*)

Crie e compile os **três fontes** do projeto:

- `STTIPLIB.PRW` — a biblioteca com `NomeCliente()`, `ProxCodigoSZ1()`, `ProxSequenSZ2()` e `DescTipoInteracao()`
- `STTIP003.PRW` — a rotina de **Contatos (SZ1)** com `mBrowse`, **legendas coloridas** e o botão “Interações”
- `STTIP004.PRW` — a rotina de **Interações (SZ2)** filtrada pelo contato selecionado

Teste: abra Contatos, selecione um e clique em **Interações** — só devem aparecer as interações daquele contato.

Exercício 3 — Gatilhos, campos virtuais e validações cruzadas (*bônus*)

Deixe o sistema “esperto”:

- **Campos virtuais** na SZ2 (`Z2_CODIGO` , `Z2_ASSUNT`) via `POSICIONE` no `X3_RELACAO`
- **Gatilhos automáticos** na SZ2:
 - `Z2_DATA` → `dDataBase` (fase 1)
 - `Z2_HORA` → `IF(INCLUI, Time(), SZ2->Z2_HORA)` (fase 3)
 - `Z2_USUAR` → `cNomUsr` (fase 1)
- **Validação cruzada** no `X3_VALID` do `Z2_CONTAT` :

```
ExistCpo("SZ1", xFilial("SZ1") + M->Z2_CONTAT, 1)
```

Teste: ao incluir uma interação, data/hora/usuário aparecem **sozinhos**; e o sistema **recusa** um contato inexistente.

Exercício 4 — Menu no SIGACOM (*bônus*)

Configure o menu (SIGACFG > Menu do Sistema) do módulo de Compras:

```
Cadastros
├── Contatos          → USER FUNCTION STTIP003
└── Interações (todas) → USER FUNCTION STTIP004B
```

STTIP004B é a versão **sem filtro** (listagem geral) — uma cópia da STTIP004 sem o **cFiltro**.

Exercício 5 — ★ Primeiro contato com erros (Harbour puro) (*obrigatório*)

Antes de blindar o sistema, treine o mecanismo num programa simples. Escreva um **.prg** que:

- Provoque **de propósito** uma divisão por zero
- Capture o erro com **BEGIN SEQUENCE / RECOVER WITH oErro**
- Exiba **oErro:Description**
- Continue** executando normalmente depois do tratamento

```
FUNCTION Main()
    LOCAL nA := 10, nB := 0, nRes
    BEGIN SEQUENCE
        nRes := nA / nB
        QOut("Resultado: " + Str(nRes))
    RECOVER WITH oErro
        QOut("Erro capturado: " + oErro:Description)
    END SEQUENCE
    QOut("O programa continua de pé!")
    RETURN NIL
```

Exercício 6 — ★ Gravação à prova de falhas (*obrigatório*)

Blinde o salvar do seu cadastro de Contatos. Crie `U_STTIP003SALVAR()` que:

- Valida `Z1_CLIENTE` e `Z1_ASSUNTO` com `IF` (usa `Break()` se faltar)
- Grava dentro de `BeginTran()` + `BEGIN SEQUENCE`, com `RecLock` / `MsUnLock`
- No `RECOVER`: faz `RollBackTran()`, mostra mensagem amigável e chama `U_GRAVARLOG()`
- Só chama `CommitTran()` se tudo deu certo

Adicione também a função `U_GRAVARLOG(cFuncao, oErro)` à sua `STTIPLIB.PRW`.

Teste: provoque um erro (ex.: cliente vazio) e confirme que **nada** é gravado pela metade e a mensagem amigável aparece.

Exercício 7 — Desafio (opcional)

Escolha **um** (ou os dois, se quiser brilhar):

7a — Integridade referencial: implemente `U_VALEXCSZ1()` que **impede excluir** um Contato (SZ1) se ele tiver Interações (SZ2) vinculadas. Dica: `ExistCpo("SZ2", xFilial("SZ2") + SZ1->Z1_CODIGO, 1)`.

7b — Executor seguro genérico: crie `U_EXECUTARSEGURO(bBloco, cMsgErro)` que recebe um bloco de código, executa-o dentro de um `BEGIN SEQUENCE`, e em caso de erro exibe `cMsgErro` e grava o log — retornando `.T.` / `.F.`.

```
// Uso:  
U_EXECUTARSEGURO({|| AbrirArquivo("dados.dbf")}, "Falha ao abrir dados")
```

Como sei que acertei?

Exercício	Está pronto quando...
5 ★	O <code>.prg</code> mostra a descrição do erro e a frase final (“continua de pé”)
6 ★	Um erro no salvar não grava nada pela metade e mostra mensagem amigável
1	SZ1 e SZ2 aparecem no Configurador com campos, índices e o domínio SX5
2	Contatos abre, e o botão “Interações” mostra só as interações do contato
3	Data/hora/usuário preenchem sozinhos e um contato inexistente é recusado
4	As duas rotinas aparecem no menu de Compras e abrem sem erro
5	O <code>.prg</code> mostra a descrição do erro e a frase final (“continua de pé”)
6	Um erro no salvar não grava nada pela metade e mostra mensagem amigável
7	(7a) não deixa excluir contato com interações · (7b) o executor loga e devolve .T./F.

✅ Fez de 1 a 6? Você construiu um **sistema profissional completo** — com CRUD relacionado, validações e tratamento de erros. O 7 é o “algo a mais” que já prepara o terreno do TCC.



Como organizar no seu repositório

Salve os exercícios desta aula na pasta `modulo-09/`, um arquivo por exercício (guia completo em `pre-curso/setup-github.md`):

```
modulo-09/  
├─ ex01-dicionario-dados.md  
├─ ex02-biblioteca-rotinas/      (vários arquivos)  
├─ ex03-gatilhos-validacoes.md  
├─ ex04-menu-sigacom.md  
├─ ex05-tratamento-erros.prg  
├─ ex06-gravacao-segura.prw  
└─ ex07-executor-seguro.prw      (desafio – opcional)
```

Depois: `git add .` → `git commit -m "modulo 09 – exercicios"` → `git push`, e cole o link da pasta na atividade do Módulo 9 no Classroom.

 **E os prints?** Vão **junto, no próprio repositório** — na pasta `modulo-09/` (ou numa subpasta `modulo-09/evidencias/`). O link que você cola no Classroom já leva os prints junto; **não precisa entregar em dois lugares** (anexar direto na atividade também vale).

 A correção automática lê o **código** (`.prw` / `.prg`) e as **respostas escritas** (`.md`), **não a imagem**. O print é a prova de que rodou — o que garante a nota é o código + a explicação. **Nunca entregue só o print**. Sem ambiente? Entrega o código comentado; sem print, sem problema. 🙌

Guia de referência rápida

```
// Buscar dado de outra tabela / checar existência
POSICIONE("SZ1", 1, xFilial("SZ1") + cChave, "Z1_NOME")
ExistCpo("SZ1", xFilial("SZ1") + cChave, 1)      // .T. se existe

// Browse com opção customizada + legendas
PRIVATE aRotina := { {"Interações", "U_STTIP004", 0, 6} }
aColors := { {"regra", "BR_GREEN"}, {".T.", "BR_RED"} }
mBrowse(1, 1, 22, 75, "SZ1", , , , , , aColors, , , , .F., , , cFiltro)

// Gravação segura: transação + lock + erro
BeginTran()
BEGIN SEQUENCE
    RecLock("SZ1", .T.)          // .T. inclui · .F. altera
    SZ1->Z1_CODIGO := M->Z1_CODIGO
    MsUnLock()
    CommitTran()
RECOVER WITH oErro
    RollBackTran()
    MsgStop("Erro: " + oErro:Description, "Erro")
    U_GRAVARLOG("Salvar", oErro)
END SEQUENCE
```

- **Documentação TDN (TOTVS):** <https://tdn.totvs.com/>